

PROYECTO INTERMODULAR



**ALONSO
DE AVELLANEDA**

Alonso de Avellaneda

Técnico Superior en Desarrollo de Aplicaciones
Multiplataforma

Clef

Documentación BBDD (Firebase)

Autor: Óscar Padrino López

Autor: Adrián Muñoz Saiz

Autor: José Manuel Perdices Hernández

Director: César Rodríguez Casero

Tutor curso: Diana Pastor Lazcano

Tutor proyecto: Alejandro De Acuña Jiménez

Alcalá de Henares, mayo de 2026

Clef



Anexo B

Contenido

1. Introducción a Firebase	3
1.1 ¿Qué es Firebase?	3
1.2 Por qué se eligió Firebase para Clef	3
1.3 Servicios utilizados.....	4
2. Firebase Authentication.....	5
2.1 Métodos de autenticación	5
2.2 Flujo de registro	5
2.3 Flujo de inicio de sesión	6
2.4 Verificación de email	6
3. Cloud Firestore.....	6
3.1 Modelo de datos NoSQL.....	6
3.2 Colección users — estructura del documento	7
3.3 Descripción de cada campo en Firebase	8
3.4 Reglas de seguridad	9
4. Cloud Functions	10
4.1 ¿Qué son y por qué se usan?	10
4.2 checkLoginIp — detección de accesos sospechosos.....	10
4.3 deleteAccount / deleteAccountHttp — borrado seguro de cuenta	11
5. Sincronización y modo offline	12
5.1 Almacenamiento local cifrado	12
5.2 Versionado optimista (campo versión)	13
5.3 Sincronización selectiva (synced).....	13
5.4 Auto-lock	14
5.5 Límite de acceso offline (7 días).....	14
6. Correos electrónicos	15
6.1 Verificación de email	15
6.2 Recuperación de contraseña	15
6.3 Alerta de acceso sospechoso.....	15
7. Seguridad del dispositivo — detección de root	16
7.1 ¿Qué es el root?	16
7.2 Cómo lo detecta Clef	16
7.3 Qué hace la app al detectarlo.....	19
8. Protección contra fuerza bruta	19
9. Recuperación con PUK.....	20

10. Biometría — Android Keystore y TEE.....	21
10.1 ¿Qué es el TEE?	21
10.2 Cómo funciona el desbloqueo biométrico en Clef	21
10.3 Invalidación automática al añadir huella nueva	22

1. Introducción a Firebase

1.1 ¿Qué es Firebase?

Firebase es una plataforma de desarrollo de aplicaciones móviles y web creada por Google. Funciona como un **Backend as a Service (BaaS)**, lo que significa que proporciona la infraestructura de servidor lista para usar, sin necesidad de gestionar servidores propios.

Entre sus características principales destacan:

- **Base de datos en tiempo real:** sincronización automática de datos entre dispositivos.
- **Autenticación integrada:** gestión de usuarios con múltiples proveedores (email, Google, etc.).
- **Funciones serverless:** ejecución de lógica de servidor sin aprovisionar infraestructura.
- **Escalabilidad automática:** la plataforma escala según la demanda sin configuración adicional.
- **Integración con el ecosistema Google:** compatibilidad nativa con Android, Google Cloud y otros servicios de Google.

1.2 Por qué se eligió Firebase para Clef

La elección de Firebase como backend de Clef se basó en los siguientes criterios:

- **Plan Blaze (pay-as-you-go):** permite usar Cloud Functions, que requieren este plan, manteniendo costes mínimos gracias a la generosa capa gratuita incluida.
- **SDK oficial para Android:** integración nativa con Android Studio y Kotlin/Java, sin necesidad de implementar una API REST manualmente.
- **Autenticación lista para usar:** Firebase Auth gestiona el ciclo completo de usuarios (registro, login, verificación de email y recuperación de la contraseña de acceso) con pocas líneas de código.

- **Firestore como base de datos flexible:** el modelo de documentos JSON encaja perfectamente con la estructura de datos de Clef, donde cada usuario tiene un único documento con su vault cifrado.
- **Cloud Functions para lógica sensible:** permite ejecutar código de servidor de confianza (como el borrado de cuentas con privilegios de administrador) sin exponer credenciales en la app.
- **Seguridad:** las reglas de seguridad de Firestore permiten garantizar que cada usuario solo puede acceder a su propio documento.

1.3 Servicios utilizados

Servicio	Uso en Clef
Firebase Authentication	Registro e inicio de sesión de usuarios mediante email/contraseña y cuenta de Google
Cloud Firestore	Almacenamiento del vault cifrado y metadatos de seguridad de cada usuario
Cloud Functions	Detección de accesos sospechosos y borrado seguro de cuentas desde el servidor
Google Cloud Secret Manager	Almacenamiento seguro de la contraseña de aplicación de Gmail usada para enviar emails de alerta

2. Firebase Authentication

2.1 Métodos de autenticación

Clef soporta dos métodos de autenticación a través de Firebase Authentication:

Método	Descripción
Email y contraseña	El usuario se registra con su dirección de email y una contraseña. Firebase almacena las credenciales de forma segura.
Google Sign-In	El usuario se autentica con su cuenta de Google mediante OAuth 2.0. Firebase gestiona el token de acceso

Es importante distinguir entre la **contraseña de Firebase** (que autentica al usuario en el servicio) y la **contraseña maestra** (que cifra el vault localmente). Son independientes y con propósitos distintos.

2.2 Flujo de registro

1. El usuario introduce su email y contraseña en la pantalla de registro.
2. Firebase crea la cuenta con `createUserWithEmailAndPassword()`.
3. Se envía automáticamente un email de verificación al usuario.
4. El usuario configura su contraseña maestra para cifrar el vault.
5. Se generan las claves criptográficas y se sube el vault inicial a Firestore.

En el caso de Google Sign-In, los pasos 1-3 se sustituyen por el flujo OAuth de Google; la contraseña maestra y el cifrado del vault se configuran igualmente a continuación.

2.3 Flujo de inicio de sesión

1. El usuario se autentica con email/contraseña o con Google.
2. Firebase devuelve un `FirebaseUser` con un **ID Token** JWT firmado.
3. La app descarga el vault cifrado desde Firestore.
4. El usuario introduce su contraseña maestra para descifrar el vault localmente.
5. La sesión queda activa en memoria hasta que se bloquea o se cierra.

2.4 Verificación de email

Firebase exige que el email del usuario esté verificado antes de poder usar la app. El flujo es el siguiente:

1. Tras el registro, Firebase envía un email con un enlace de verificación.
2. La app comprueba `firebaseUser.isEmailVerified()` en cada inicio de sesión.
3. Si el email no está verificado, se muestra una pantalla de aviso y se ofrece reenviar el email.
4. Una vez verificado, el usuario puede acceder normalmente.

3. Cloud Firestore

3.1 Modelo de datos NoSQL

Cloud Firestore es la base de datos principal de Clef. A diferencia de las bases de datos relacionales, Firestore organiza los datos en colecciones y documentos:

- Una colección es un contenedor de documentos (equivalente a una tabla).
- Un documento es un conjunto de pares clave-valor en formato JSON (equivalente a una fila), con soporte para tipos como cadenas, números, booleanos, arrays y subcolecciones.

3. Cloud Firestore

- No existen joins ni esquemas fijos; cada documento puede tener campos distintos.

En Clef la estructura es muy simple: una única colección `users` donde cada documento corresponde a un usuario.

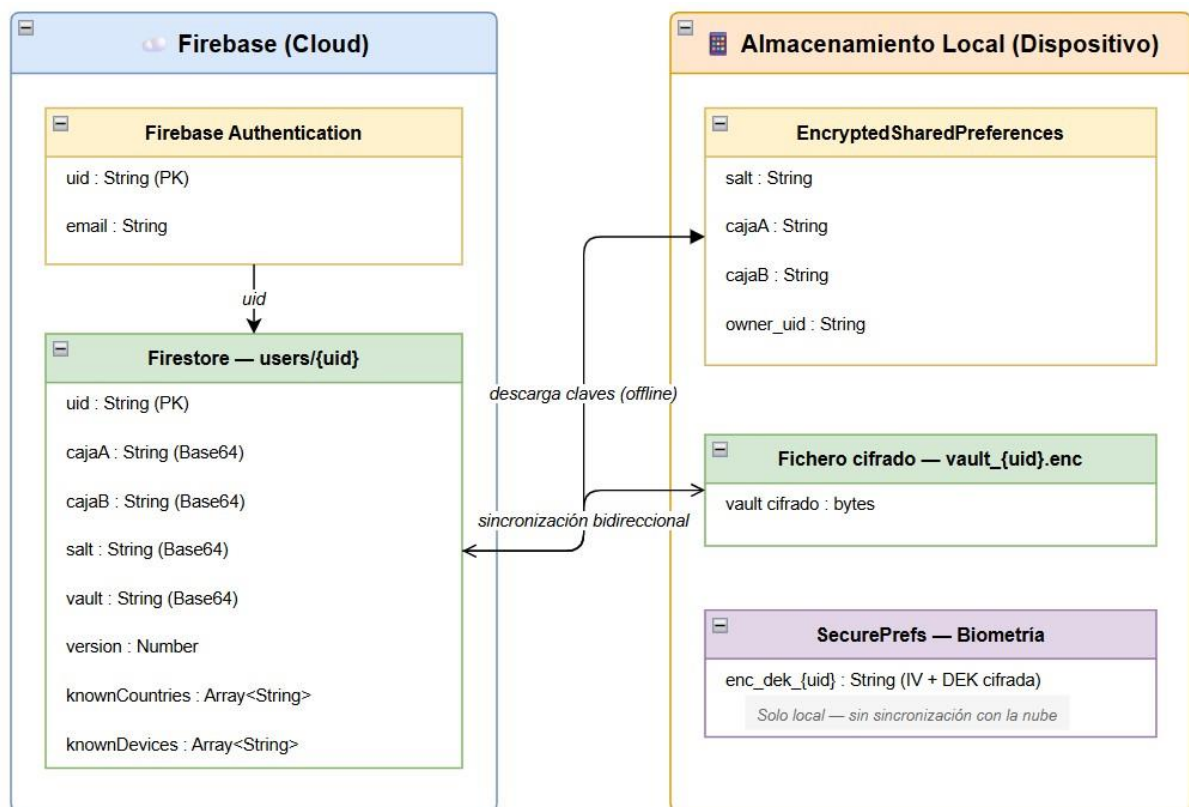
3.2 Colección users — estructura del documento

Cada documento está identificado por el **UID de Firebase Authentication** del usuario y contiene su vault cifrado junto con los metadatos de seguridad.

Formato en FireBase	Ejemplo real de FireBase de lo que contiene el usuario
<pre>users/ └─ {uid}/ └─ cajaA └─ cajaB └─ knownCountries └─ knownDevices └─ salt └─ vault └─ version</pre>	<pre>{ "cajaA": "xxxxxxxxxx/xxxx/xx", "cajaB": "xxxxxxxxxx/xxxx/xx", "knownCountries": ["ES", "FR"], "knownDevices": ["Samsung Galaxy S24", "Google Pixel 9"], "salt": "xxxxxxxxxx/xxxx/xx", "vault": "xxxxxxxxxx/xxxx/xx", "version": 1 }</pre>

Nota sobre cifrado: Firebase cifra todos los datos en tránsito con **TLS** y en reposo con **AES-25**. Clef añade además una capa de cifrado de aplicación: el vault llega a Firestore ya cifrado, de modo que ni Google puede leer las contraseñas del usuario.

Diagrama entidad-relación lógico de la base de datos de Clef:



3.3 Descripción de cada campo en Firebase

Campo	Tipo	Descripción
cajaA	String (Base64)	DEK (clave de cifrado del vault) cifrada con la contraseña maestra del usuario.
cajaB	String (Base64)	DEK cifrada con el PUK (código de recuperación). Permite recuperar el vault si el usuario olvida su contraseña maestra.
knownCountries	Array<String>	Lista de países (código ISO 3166-1 alpha-2) desde los que el usuario ha iniciado sesión. La IP no se almacena; solo se usa para obtener el país.
knownDevices	Array<String>	Lista de modelos de dispositivo conocidos del usuario.
salt	String (Base64)	Valor aleatorio usado en la derivación de claves mediante PBKDF2. Único por usuario, generado en el registro.

vault	String (Base64)	Vault completo serializado a JSON y cifrado con la DEK. Contiene todas las credenciales del usuario.
version	Number	Número de versión del vault. Se incrementa en cada actualización para detectar conflictos de sincronización.

3.4 Reglas de seguridad

Las reglas de seguridad de Firestore garantizan que cada usuario solo puede leer y escribir su propio documento. Ningún usuario puede acceder al documento de otro.

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Solo el usuario autenticado puede leer y escribir sus propios datos.
    // Ruta: /users/{uid} donde uid es el ID único de Firebase Auth.
    match /users/{uid} {
      // request.auth != null → debe estar autenticado (no anónimo)
      // request.auth.uid == uid → solo puede acceder a SU documento, no al de otros
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
  }
}

```

4. Cloud Functions

4.1 ¿Qué son y por qué se usan?

Cloud Functions es el servicio serverless de Firebase que permite ejecutar código de servidor en respuesta a eventos o llamadas desde la app, sin necesidad de gestionar infraestructura propia.

En Clef se usan por dos motivos concretos que no se pueden resolver desde el cliente Android:

- **Seguridad con privilegios de administrador:** borrar una cuenta de Firebase Auth requiere el Admin SDK, que solo puede usarse en el servidor para evitar exponer credenciales con permisos elevados en la app.
- **Detección de accesos sospechosos:** la IP del cliente solo es fiable si la lee el servidor; desde la app podría manipularse.

Durante la inicialización del proyecto de funciones, Firebase ofrece tres lenguajes: **Python**, **TypeScript** y **JavaScript**. Se eligió JavaScript porque cuenta con mayor cantidad de documentación y ejemplos disponibles en la web, lo que facilita el desarrollo y la resolución de problemas.

Todas las funciones están desplegadas en la región **us-central1** del proyecto **clef-efe86**.

4.2 checkLoginIp — detección de accesos sospechosos

Esta función se llama automáticamente desde la app tras cada inicio de sesión exitoso.

Tipo: Callable (llamada directa desde el SDK de Firebase en Android)

4. Cloud Functions

Parámetro de entrada:
<pre>{ "device": "Samsung Galaxy S24" }</pre>

Lógica:

1. Captura la IP real del cliente desde el contexto de la petición.
2. Consulta la API de geolocalización ip-api.com para obtener el país.
3. Comprueba si el dispositivo y el país ya están en knownDevices y knownCountries del documento del usuario en Firestore.
4. Si es el primer login, añade dispositivo y país sin alertar.
5. Si el dispositivo o el país son nuevos, los añade a Firestore y envía un email de alerta de seguridad al usuario.

Respuesta:
<pre>{ "isNew": true }</pre>

Si la llamada tiene éxito, la app navega a la pantalla principal independientemente del valor de isNew. El aviso al usuario se realiza exclusivamente por **email**; no hay ningún mensaje en la propia app para no alertar al posible sospechoso.

4.3 deleteAccount / deleteAccountHttp — borrado seguro de cuenta

El borrado de cuenta requiere ejecutarse en el servidor porque Firebase Auth exige re-autenticación reciente para user.delete(), y los usuarios de Clef no conocen su contraseña de Firebase (solo la maestra).

- **deleteAccount:** Callable
- **deleteAccountHttp:** HTTP (usada como alternativa si la versión callable falla)

Autenticación: La app obtiene el ID Token con `getIdToken(true)` y lo envía como Bearer en la cabecera Authorization.

Operaciones que realiza la función:

1. Verifica el ID Token con el Admin SDK.
2. Borra el documento del usuario en Firestore.
3. Borra la cuenta de Firebase Authentication.

En la app, tras la respuesta exitosa:

1. Se elimina el vault cifrado del almacenamiento local.
2. Se limpia la caché de claves.
3. Se cierra la sesión y se navega a la pantalla de login.

5. Sincronización y modo offline

5.1 Almacenamiento local cifrado

Clef no depende de tener conexión a internet para funcionar. El vault cifrado se guarda en el almacenamiento interno del dispositivo en un fichero con el nombre `vault_{uid}.enc`, donde `{uid}` es el UID de Firebase del usuario. Usar el UID en el nombre del fichero evita colisiones si se cambia de cuenta en el mismo dispositivo.

Además, las claves necesarias para descifrar el vault (``salt``, ``cajaA``, ``cajaB``) se guardan en caché usando **EncryptedSharedPreferences** de Android, protegidas por el Android Keystore. Esto permite que el usuario pueda iniciar sesión y descifrar su vault sin conexión, usando solo su contraseña maestra.

5.2 Versionado optimista (campo versión)

Para evitar conflictos cuando el vault se modifica desde varios dispositivos, Clef usa **control de concurrencia optimista** basado en el campo version del documento de Firestore.

El flujo es el siguiente:

1. Al descargar el vault, la app guarda la versión actual (version = N).
2. Al subir cambios, la app envía la versión esperada junto con la nueva (N → N+1).
3. Si en Firestore la versión ya no es "N" (otro dispositivo subió cambios antes), la operación se rechaza y la app descarga la versión más reciente antes de reintentar.

Esto garantiza que nunca se sobrescriben cambios sin querer.

5.3 Sincronización selectiva (synced)

Cada credencial almacenada dentro del vault JSON tiene un campo `synced` (booleano) que indica si debe subirse a Firebase o mantenerse solo en local. Este campo no aparece en el documento de Firestore directamente, sino dentro del JSON cifrado que forma el vault.

- **synced = true:** la credencial se incluye en las subidas a Firestore.
- **synced = false:** la credencial existe solo en el dispositivo y nunca sale de él.

Esto permite al usuario tener contraseñas que prefiere no almacenar en la nube, manteniendo el control total sobre qué datos se sincronizan.

5.4 Auto-lock

Cuando la app pasa a segundo plano, se inicia un temporizador de inactividad. Si el usuario no vuelve antes de que expire, la sesión se bloquea automáticamente.

El bloqueo no es lógico sino físico: la DEK (clave de cifrado del vault) se sobrescribe con ceros en memoria usando `Arrays.fill(dek, 0x00)` y el vault se elimina de RAM. Aunque alguien hiciera un volcado de memoria del proceso justo después del bloqueo, encontraría ceros donde estaba la clave.

El tiempo de bloqueo es configurable por el usuario: 1, 5, 15, 30 minutos o Nunca. Tras el bloqueo, la siguiente apertura de la app lleva a `UnlockActivity`, donde se pide la contraseña maestra o la huella dactilar para descifrar de nuevo el vault.

5.5 Límite de acceso offline (7 días)

Cuando la app arranca sin conexión a internet, `checkLoginIp` no puede ejecutarse. En ese caso la app sigue el siguiente criterio:

Situación	Comportamiento
Sin verificación previa (primer uso)	Deja pasar y guarda el timestamp
Última verificación hace menos de 7 días	Deja pasar con aviso de modo offline
Sin verificación durante más de 7 días	Bloquea el acceso hasta que haya conexión

El timestamp de la última verificación exitosa se guarda localmente. Este límite existe para evitar que un dispositivo robado pueda usarse indefinidamente sin conexión al servidor.

6. Correos electrónicos

Clef envía tres tipos de correos a los usuarios. Los dos primeros son gestionados directamente por **Firestore Authentication**; el tercero es enviado por una **Cloud Function** usando **Nodemailer** con una cuenta Gmail dedicada (security.clef@gmail.com), cuya contraseña de aplicación se almacena de forma segura en **Google Cloud Secret Manager**.

6.1 Verificación de email

- Remitente: `security.clef@gmail.com`
- Cuándo se envía: Automáticamente al crear una cuenta nueva.
- Propósito: Confirmar que el email introducido pertenece al usuario antes de permitirle usar la app.

6.2 Recuperación de contraseña

- Remitente: `security.clef@gmail.com`
- Cuándo se envía: Cuando el usuario pulsa "¿Olvidaste tu contraseña?" en la pantalla de login.
- Propósito: Permitir al usuario restablecer su contraseña de acceso a Firestore. No afecta al vault cifrado, que sigue protegido por la contraseña maestra.

6.3 Alerta de acceso sospechoso

- Remitente: `security.clef@gmail.com`
- Asunto: Nuevo acceso sospechoso detectado - Clef
- Cuándo se envía: Cuando la Cloud Function checkLoginIp detecta un inicio de sesión desde un dispositivo o país no reconocido.

Contenido del correo:

- Dispositivo desde el que se ha accedido
- Ciudad detectada por geolocalización de IP
- Motivo de la alerta (dispositivo nuevo o país nuevo)
- Fecha y hora del acceso (zona horaria Europe/Madrid)
- Aviso para cambiar la contraseña maestra si el acceso no fue el propio usuario

7. Seguridad del dispositivo — detección de root

7.1 ¿Qué es el root?

En Android, el root (o acceso superusuario) es el proceso de obtener privilegios de administrador completos sobre el sistema operativo del dispositivo. Un dispositivo con root ha roto las restricciones de seguridad impuestas por Android, lo que permite a cualquier app con esos privilegios acceder al almacenamiento privado de otras aplicaciones, modificar archivos del sistema o interceptar datos en memoria.

Para una app de gestión de contraseñas esto supone un riesgo crítico: un atacante con acceso root podría leer el vault cifrado directamente del disco o extraer las claves de la memoria mientras la sesión está activa.

7.2 Cómo lo detecta Clef

Clef ejecuta la detección al arrancar la app mediante la clase RootDetector. Los checks se dividen en dos grupos:

Indicios definitivos (uno solo es suficiente para bloquear):

Check	Descripción
Binario “ su ” en rutas estándar	Comprueba si existe <code>`/system/bin/su`</code> , <code>`/system/xbin/su`</code> , <code>`/sbin/su`</code> o <code>`/su/bin/su`</code>
Apps de root instaladas	Detecta la presencia de Magisk, SuperSU, KingRoot, Kingo Root, Superuser y otras
Partición del sistema modificable	Ejecuta <code>`mount`</code> y comprueba si <code>`/system`</code> está montado en modo escritura (<code>`rw`</code>)

Indicios sospechosos (se necesitan 3 o más para bloquear):

Check	Descripción
Binario “ su ” en rutas inusuales	Rutas como <code>`/data/local/su`</code> o <code>`/data/local/xbin/su`</code>
Claves de firma del sistema	<code>`Build.TAGS`</code> contiene <code>`test-keys`</code> o <code>`dev-keys`</code>
Modo debug activo	Propiedad del sistema <code>`ro.debuggable = 1`</code>
Sistema no seguro	Propiedad del sistema <code>`ro.secure = 0`</code>

Cada uno de estos indicios por separado puede aparecer en dispositivos legítimos sin root: un desarrollador puede tener el modo debug activo, algunas ROMs personalizadas usan ``test-keys`` sin ser maliciosas, y ciertas rutas sospechosas

pueden existir sin privilegios reales. Bloquear con un único indicio generaría demasiados falsos positivos e impediría el acceso a usuarios legítimos. Exigir 3 o más de forma simultánea hace muy improbable que sea una coincidencia, apuntando a un dispositivo realmente comprometido.

Ejemplos de dispositivos o situaciones que pueden activar indicios sospechosos sin estar necesariamente rooteados:

Dispositivo / Situación	Indicios típicos
Xiaomi con ROM de desarrollador o beta	test-keys en versiones beta de MIUI/HyperOS
OnePlus con OxygenOS Open Beta	test-keys en builds de prueba
Dispositivos con LineageOS u otras ROMs personalizadas	test-keys por defecto al compilar desde AOSP
Marcas chinas de gama baja (Doogee, Ulefone, Oukitel)	Algunas unidades de fábrica incluyen test-keys o ro.secure=0 en sus ROMs de serie
Cualquier móvil con bootloader desbloqueado	Normalmente lleva test-keys tras flashear firmware no oficial
Cualquier móvil con opciones de desarrollador activadas	ro.debuggable=1 si se ha habilitado la depuración USB

Para evitar bloquear el hilo principal, los comandos del sistema (``mount``, ``getprop``) se ejecutan en hilos de fondo con un **timeout de 500 ms** cada uno.

7.3 Qué hace la app al detectarlo

La comprobación ocurre en “SplashActivity” antes de cualquier otra acción. Si se detecta root:

1. Se muestra un diálogo **no cancelable** con el motivo del bloqueo.
 - a. Si el indicio es claro (root confirmado): título **"Dispositivo no seguro"**
 - b. Si son varios indicios sospechosos: título **"Múltiples indicios de riesgo"**
2. El único botón disponible es **"Cerrar app"**, que termina la actividad.
3. La app no llega a inicializar Firebase ni a cargar ningún dato del usuario.

8. Protección contra fuerza bruta

“BruteForceGuard” protege el vault ante intentos repetidos de contraseña maestra incorrecta. Cada fallo desencadena un bloqueo de duración creciente antes de permitir otro intento:

Intento fallido	Espera obligatoria
1.º	10 segundos
2.º	30 segundos
3.º	60 segundos
4.º	120 segundos
5.º	240 segundos

Tras **5 fallos consecutivos** el comportamiento varía según la pantalla:

- Pantalla de contraseña maestra (`UnlockActivity`): redirige al flujo de recuperación con PUK.
- Pantalla de recuperación con PUK (`RecoverVaultActivity`): cierra sesión completamente y navega a la pantalla de login, impidiendo seguir probando códigos.

Los contadores de intentos fallidos se guardan en **EncryptedSharedPreferences** (no en SharedPreferences sin cifrar), lo que impide que un atacante con acceso ADB los resetee para saltarse el bloqueo. Los contadores son por UID, de modo que bloquear una cuenta no afecta a otras en el mismo dispositivo.

9. Recuperación con PUK

El PUK (Personal Unblocking Key) es un código de 32 caracteres hexadecimales generado una sola vez en el registro y mostrado al usuario para que lo guarde en un lugar seguro. Sirve para recuperar el vault si el usuario olvida su contraseña maestra.

Flujo de recuperación:

1. El usuario pulsa "Usar PUK" en la pantalla de login.
2. La app descarga de Firestore: `salt`, `cajaB` y `vault`.
3. El usuario introduce su PUK.
4. `PBKDF2(PUK, salt, 230.000 iter): `KEK_PUK`
5. `AES-GCM-Descifra(cajaB, KEK\_PUK)`: **DEK** — la misma clave que siempre ha cifrado el vault.
6. `AES-GCM-Descifra(vault, DEK)`: vault descifrado.
7. El usuario elige una contraseña maestra nueva.
8. Se genera un **nuevo PUK** aleatorio.
9. Se recalculan `cajaA` y `cajaB` con las nuevas claves y se suben a Firestore en una escritura atómica.

10. El nuevo PUK se muestra al usuario una única vez.

Invalidación criptográfica del PUK viejo: el PUK anterior no se "marca como usado" en ninguna lista. Queda invalidado porque la `cajaB` que descifraba ya no existe — ha sido sobrescrita por la nueva. Es invalidación por destrucción, no por lógica de aplicación.

10. Biometría — Android Keystore y TEE

10.1 ¿Qué es el TEE?

El TEE (Trusted Execution Environment) es un entorno de ejecución aislado que existe dentro del SoC (procesador) del dispositivo, separado del sistema operativo Android. Las claves guardadas en el TEE nunca salen de él: cuando una app quiere usarlas, el sistema operativo le devuelve un objeto "Cipher" ya inicializado por el hardware, y la operación criptográfica ocurre dentro del chip, no en la RAM del proceso.

Esto significa que aunque alguien tenga root y vuelque toda la memoria del proceso de Clef, no encontrará la clave del Keystore. Esa clave vive en otro mundo.

10.2 Cómo funciona el desbloqueo biométrico en Clef

Activar biometría:

1. El Android Keystore genera una clave AES-256-GCM dentro del TEE. La clave nunca sale.
2. El usuario se autentica con huella dactilar.
3. El TEE expone un "Cipher" inicializado para cifrar.
4. Se cifra la DEK con ese "Cipher": DEK_cifrada_por_keystore.
5. IV (12 bytes) y DEK_cifrada_por_keystore se concatenan y se guardan como un único string Base64 en las preferencias seguras.

Desbloquear con biometría:

1. Se lee el string Base64 combinado de las preferencias seguras y se extraen el IV (primeros 12 bytes) y la “DEK_cifrada_por_keystore” (el resto).
2. El usuario pone el dedo en el sensor.
3. El TEE expone un `Cipher` inicializado para descifrar.
4. “Cipher.doFinal(DEK_cifrada_por_keystore)”: **DEK en RAM.**
5. La app descifra el vault con la DEK y la sesión queda activa.

10.3 Invalidación automática al añadir huella nueva

La clave del Keystore se crea con `setInvalidatedByBiometricEnrollment(true)`. Esto significa que si alguien añade una huella nueva al dispositivo (por ejemplo, tras robarlo), la clave se **autodestruye automáticamente**. La `DEK\_cifrada\_por\_keystore` deja de poder descifrarse y la única vía para acceder al vault pasa a ser la contraseña maestra.